

LG Soft –Placement Prep

Compiled from student-reported interview experience notes | Emertxe Placement Prep

Round Structure (1st Round – MCQ)

Section	No. of Questions	Type
Sec 1	10	Conceptual
Sec 2	10	Predict the Output
Sec 3	5	Find the Error in the Code

C++ Topics Covered

OOP Fundamentals

- Move constructor
- Non-trivial destructor
- Abstract class
- Access specifiers
- Virtual functions / inheritance

Exception Handling

- `catch(const std::exception&e)` style handling

Modern C++ / Type System

- `constexpr`
- `std::variant`
- `std::visit`
- `std::optional`
- `std::move`

Smart Pointers

- `std::shared_ptr`
- `std::make_shared`
- `use_count`

STL Algorithms & Functors

- `std::accumulate`
- `count_if`
- Lambda functions
- `std::greater<int>`
- `vector::erase()`

MCQ / Predict-the-Output Questions

Q1.

Given `vector<int> speeds{80,120,90,100,85};` and a function `check_speed(int s)` that throws `std::out_of_range` if `s > 100`. Inside a single try block, the calls `check_speed(100);` `check_speed(150);` `check_speed(120);` are made, caught by `catch(const std::out_of_range& e)`. What is the output?

Answer: Caught: speed exceeded followed by Done

Note: the original handwritten snippet had the final cout statement outside the function scope referencing a local parameter — corrected here so the question is compilable as written.

Q2.

`vector<int> v{2,1,3,5,4}; sort(v.begin(), v.end(), greater<int>());` What is the resulting order of v?

- A) 1 2 3 4 5
- B) 5 4 3 2 1
- C) Undefined
- D) Compile error

Answer: B — 5 4 3 2 1

Q3.

What does `std::move(x)` actually do internally?

- A) Physically relocates the object in memory
- B) Casts x to an rvalue reference, enabling move semantics if a move constructor/assignment exists
- C) Deep-copies x
- D) Deletes x

Answer: B

Q4.

Base class declares virtual functions but has a non-virtual destructor. A Derived object is deleted through a Base pointer. What happens?

- A) Compile-time error
- B) Only Base's destructor runs — Derived's destructor is skipped, causing undefined behavior / resource leaks
- C) No difference in behavior
- D) Guaranteed crash

Answer: B

Note: original phrasing implied a compiler error — corrected, since this is silent undefined behavior at runtime, not a compile-time failure.

Q5.

Which access specifier allows a derived class to directly access private members of its base class?

- A) public
- B) protected
- C) private
- D) None — private members are never directly accessible from a derived class

Answer: D

Common misconception to watch for: students often pick B (protected).

Q6.

Find the error:

```
for (it = v.begin(); it != v.end(); ++it)
    if (*it == 3) it = v.erase(it);
```

Answer: erase(it) already returns the next valid iterator; the loop's ++it then skips past it, silently omitting the following element. Fix: use continue after reassigning it, or restructure as a while-loop.

Q7.

vector<int> v{10,20,30}; for (int &x : v) x = x*2; Final contents of v?

Answer: 20 40 60

Programming / Coding Question (kept separate — not MCQ format)

Rod-cutting combinatorics. A rod of length N must be cut into exactly R pieces, each piece a positive integer length (pieces need not be equal). Count the number of distinct ways to make these cuts. Return the result modulo 1,000,000,007.

Examples:

$N = 10, R = 10 \rightarrow$ only 1 way: 1,1,1,1,1,1,1,1,1,1

$N = 10, R = 2 \rightarrow$ 9 ways: (1,9) (2,8) (3,7) (4,6) (5,5) (6,4) (7,3) (8,2) (9,1)

Approach: This is a stars-and-bars composition-counting problem. The number of ways to write N as an ordered sum of R positive integers is $C(N-1, R-1)$. Compute via modular binomial coefficient (precomputed factorials + modular inverse) for efficiency at scale, mod 1,000,000,007.

Face-to-Face (F2F) Interview Questions

● C++ (CPP)

► 1. Basics & C vs C++

- Difference between C and C++
- OOP vs POP difference
- What is a class?
- Class vs Object — difference with real-time example
- Namespace — what it is and why used (C++ context)
- Struct difference between C and C++
- Class vs Structure difference (with memory layout)
- Can OOP concepts be used in structures?
- SDLC
- What is G++
- Second largest element in array (C++ implementation)

► 2. OOP — Four Pillars

- OOP concepts explanation with examples + code
- OOP with real-time example
- How to include OOP concepts in a project (practical application)

- Encapsulation — in-depth explanation
- Abstraction and Abstract class
- Inheritance — types with block diagram and examples
- Inheritance — sample code questions
- Polymorphism — depth (compile time + runtime)
- Why use polymorphism — real-life example and justification (not just definition)
- ▶ **3. Constructors, Destructors & Memory**
 - Constructors (types and usage)
 - Destructors
 - Deep copy vs Shallow copy
 - How do you dynamically allocate an object in C++
 - Dynamic memory allocation in C++ with code
 - Memory leakage — what it is and how to prevent
 - How to check if memory is allocated in C++
- ▶ **4. Pointers & References**
 - Reference variable
 - Difference between pointer and reference variable
 - Single pointer vs double pointer — difference
 - How to modify an integer using double pointer
- ▶ **5. Polymorphism — Virtual Functions & Internals**
 - Virtual functions
 - Virtual and override keywords in C++ — why virtual keyword is used
 - How virtual keyword works internally
 - Pure virtual functions
 - Abstract classes — implementation
 - Abstract class with code
 - Runtime polymorphism — concept with code
 - Runtime polymorphism — vtable, vptr
 - VTable and VPtr — in-depth
 - Object slicing
- ▶ **6. Function Overloading & Overriding**
 - Function overloading with example
 - Function overloading vs overriding difference
- ▶ **7. Modern C++ Features (C++11 / C++17)**
 - Lambda functions in C++
 - constexpr — usage and difference from const
 - Structured bindings (C++17)

- `is_integral` — type traits
- Tuple — usage in C++
- Smart pointers
- Design patterns in C++
- ▶ **8. *Standard Template Library (STL)***
 - Templates — function and class templates
 - STL — `map`, `unordered_map`, `vector` (concepts + differences)
 - STL — `set`, `vector`, `queue`, `stack`, `heap`, template classes
 - STL — `map`, `pair`
 - STL Algorithms — `all_of` and similar
 - String — complexity, `append()` method
- ▶ **9. *Multithreading & Synchronisation***
 - Thread creation using C++
 - Semaphore and Mutex creation using C++
 - Deadlock (concept + prevention + overcoming)

● **Advanced C (Adv C)**

- ▶ **1. *Pre-processors***
 - What is `#include` and how it works
 - What is a header file
 - What is the preprocessing stage (compilation pipeline)
 - What is GCC
- ▶ **2. *Data Types, Variables & Keywords***
 - Range of `int`
 - need of `const` in C
 - Volatile keyword — purpose, use cases in embedded context
- ▶ **3. *Storage Classes & Memory Layout***
 - Static, extern, and all storage classes
 - Global variable vs static variable — difference
 - Where do static variables reside during runtime (memory segment)
 - Memory segments — text, data, BSS, heap, stack (explain each)
 - Difference between static and dynamic memory
 - Stack vs Heap memory
 - Memory leakage — explanation and prevention
- ▶ **4. *Pointers, Structures & Unions***
 - Function pointer — concept and usage

- Structure vs Union — difference, memory layout, use cases

► **5. Arrays & Strings**

- 2D arrays — concept and operations
- What is a string in C
- Reverse string (with and without temp variable)
- Reverse string — Reverse the alternate characters in the string
- Reverse words in a string — e.g. 'My name is Sreeraj' → 'Sreeraj is name My' (write code + test cases)
- String concatenation and remove duplicates (program)
- Compare two strings without case sensitivity (write code)
- strstr — implementation
- atoi program
- String Palindrome program

► **6. Bit Manipulation**

- Bitwise operators — types and when they are used
- Number of set bits counting (bit manipulation)
- Check whether a bit at a particular position is set (write code)

► **7. Recursion & Pattern Problems**

- Print numbers in a given range without using loops (recursion)
- Print numbers in descending order using recursion
- Fibonacci series using recursion
- Pattern printing questions

► **8. Array Problems (Coding)**

- Arrays — concepts and operations
- Second largest element in array
- Find missing element in an array
- Remove duplicates and replace with missing number in array
- Reverse array
- Swap two numbers
- Random number generation and inserting into array
- Print missing number and repeated number in an array
- Time complexity analysis for given code snippets
- Find increasing elements in array in $O(n)$
- Find next greater element for each element in $O(n)$
- Sum of array elements equal to key in $O(n)$
- Container with maximum water (LeetCode #11)

► **9. Projects**

- Address book project — write function to load structure data into file
- Steganography project – block diagram/flowchart and usage

► **10. Code Analysis & Optimisation**

- Optimised C code — time complexity considerations
- Write whether runtime error or compile time (code prediction)

⚠ Note: Volatile + Storage classes are high-frequency across multiple students. Students must know volatile specifically in the context of hardware registers and ISR variables, not just the textbook definition.

Data Structures (DS)

► **1. Linked List**

- Linked list — write code and implement any two functions (in-depth)
- Single linked list — delete node program
- Remove node in single linked list with code + iteration explanation
- Find middle node in linked list (write code)
- Find nth node from the end of a linked list (write code)
- Add element in middle of Singly Linked List
- Add element in middle of Doubly Linked List
- Delete middle node in Singly Linked List
- Delete middle node in Doubly Linked List
- Delete node at (middle-1) and (middle+1) positions
- Sort a linked list program
- Sort a doubly linked list (program)
- Find loop in a linked list (Floyd's cycle detection)

► **2. Searching Algorithms**

- Searching algorithms (general)
- Binary search — write code, explain each iteration
- Binary search time complexity

► **3. Sorting Algorithms**

- Sorting techniques (general)
- Sorting algorithms — types and comparison (including tree sort, heap sort)
- Quick sort logic

► **4. Hashing**

- Hashing — concept and use cases

► **5. Stack & Queue**

- Stack and Queue (concept + implementation using array)

- Stack vs Queue difference
- Stack implementation
- Circular Queue program
- Check whether parentheses match using stack (program)
- Implement undo-redo system (project + code)
- ▶ **6. Trees**
 - What is a tree — structure, working, and types (in-depth)
 - Tree traversal types — inorder, preorder, postorder
- ▶ **7. Graphs**
 - DFS and BFS — concept, difference, and awareness
- ▶ **8. Project-based Coding**
 - Explain Inverted Search Project
 - Explain APC Project

⚠ Note: *Linked List middle operations are the most repeated DS pattern. Students must practice all 4 variants: add/delete × single/double.*